

PYCON US 2026 - 30 MIN

# Breaking the Speed Limit

Fast statistical models with Python 3.14, Numba, and JAX  
— from a statistician's workflow.

Wenxin Jiang and Jian Yin - Department of Biostatistics, City University of Hong Kong

## Validate → accelerate → scale

Define the validation target, check it with simulation, then speed up the bottleneck.

## Simulation first

Known targets before runtime charts.

## Python interface

NumPy, Numba, Python 3.14, and JAX (A100) solve different bottlenecks.

# For statisticians, speed comes after validation

Optimization starts after the validation target is defined, stressed, and checked.

## Statistical specification evolves

Simulation exposes assumptions, hard cases, and expected behavior.

## Reference code defines the target

Readable Python or NumPy makes the validation target executable.

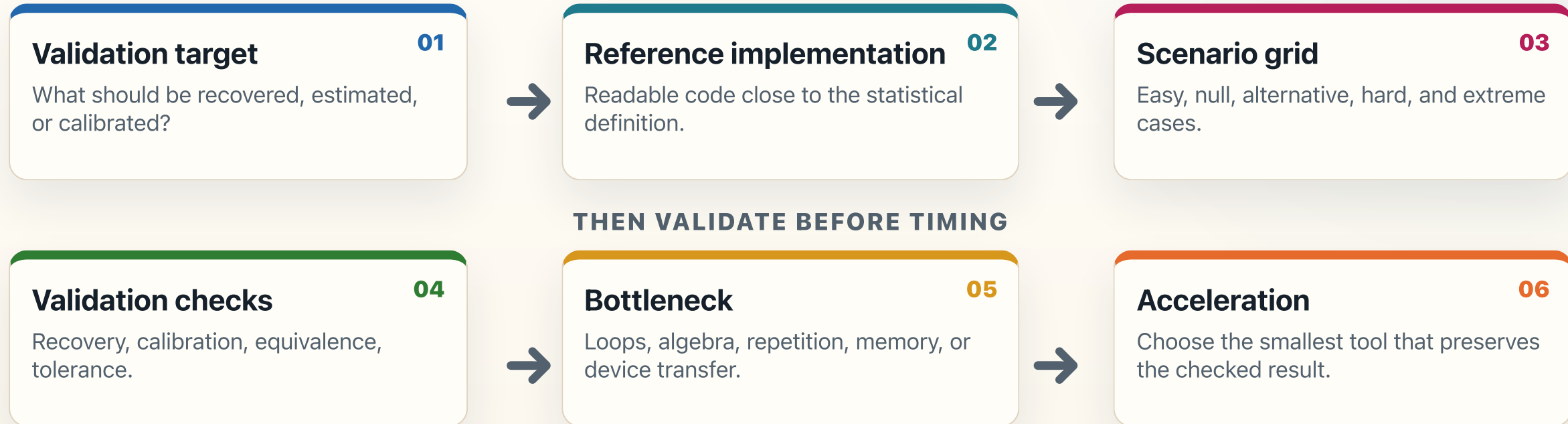
## Speed is conditional

Optimized paths must answer the same statistical question across controlled scenarios.

A speedup only counts if the optimized code answers the same statistical question.

# Simulation creates the validation target

In real data analysis, ground truth is often unavailable.



# Simulation-driven statistical computing is statistical CI

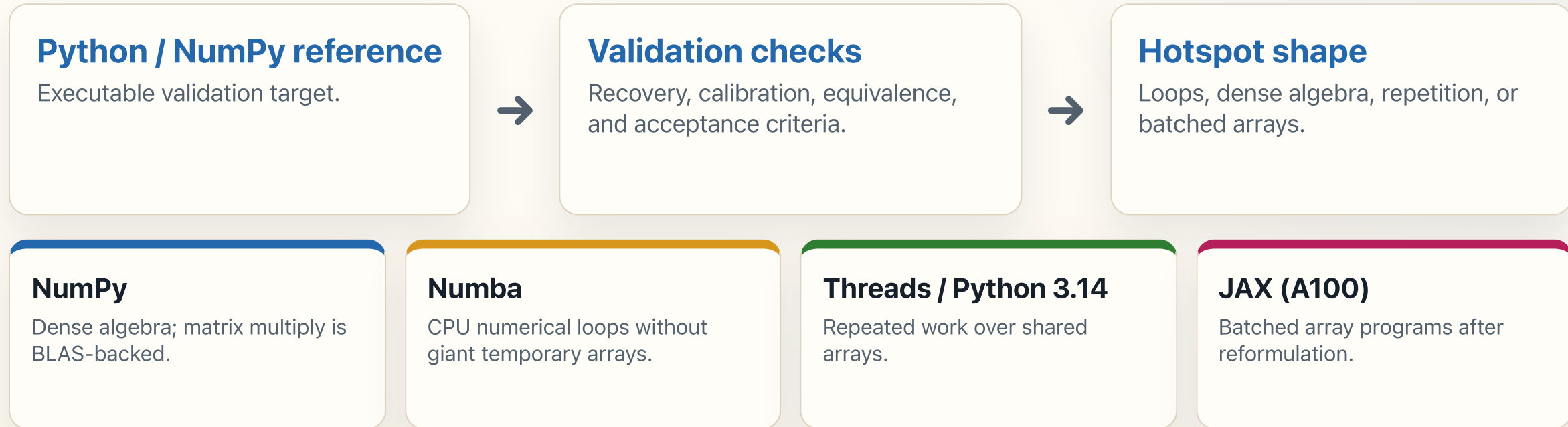
The testing instinct is familiar; the expected behavior is statistical, not one fixed scalar.

## How simulation maps to software testing

|                  |                                                |
|------------------|------------------------------------------------|
| Unit tests       | Fixed-seed reference equivalence.              |
| Property tests   | Null invariants and calibration.               |
| Stress tests     | Imbalance, outliers, high dimension.           |
| Load tests       | Scale N, d, K, p, R, workers, and batch sizes. |
| Regression tests | Acceptance criteria continue to pass.          |

# Python is the control plane; the hotspot can move to the right engine

Keep the model readable, then move the measured hotspot into the smallest engine that preserves the validation target.



## Two workloads, two statistical computing shapes

### **k-means = iterative model fitting**

assign → update centroids → repeat

Each step depends on the previous centroids.

**distance loops**

**temporaries**

**compiled kernels**

### **permutation test = resampling inference**

shuffle labels → compute statistic → repeat

Independent draws become expensive across many features.

**shared arrays**

**batching**

**W @ X**

# Evidence ladder: validation, scale, acceleration

## MacBook Air

Validation tier: controlled simulation and reference equivalence.



## Linux server CPU

Scale tier: larger shapes, parallelism, and memory accounting.



## A100 GPU

Accelerator tier: only when the pipeline is batched and device-resident.

## Correctness before runtime

Every timing row carries a validation status.

## Warm timing plus memory

Cold calls, warm medians, and RSS are logged separately.

## Unavailable rows are explicit

JIT, GPU sync, OOM, and interpreter state are labeled separately.

## WORKLOAD 1

# k-means as iterative model-fitting pressure

---

Same data and initialization; vary geometry and scale.

# How k-means moves

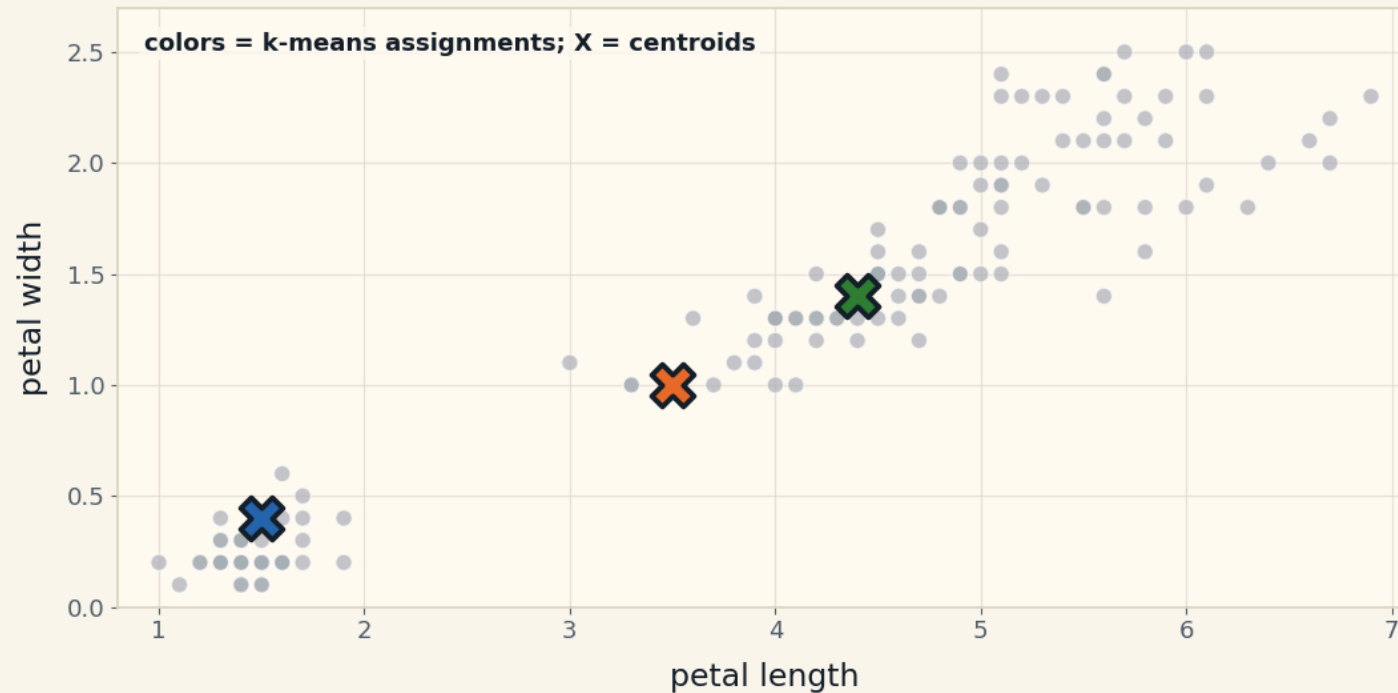
Each flower is a point; k-means uses petal length and petal width to form 3 clusters.

## Iris petal measurements

x = petal length, y = petal width

Iteration 0 / random initial centroids

fixed initial rows: 15, 60, 75



## One Lloyd run

1. **initialize:** choose 3 random flowers as centroids
2. **assign:** send each flower to the nearest centroid
3. **update:** recompute centroids from assigned flowers
4. **stop:** when assignments no longer change



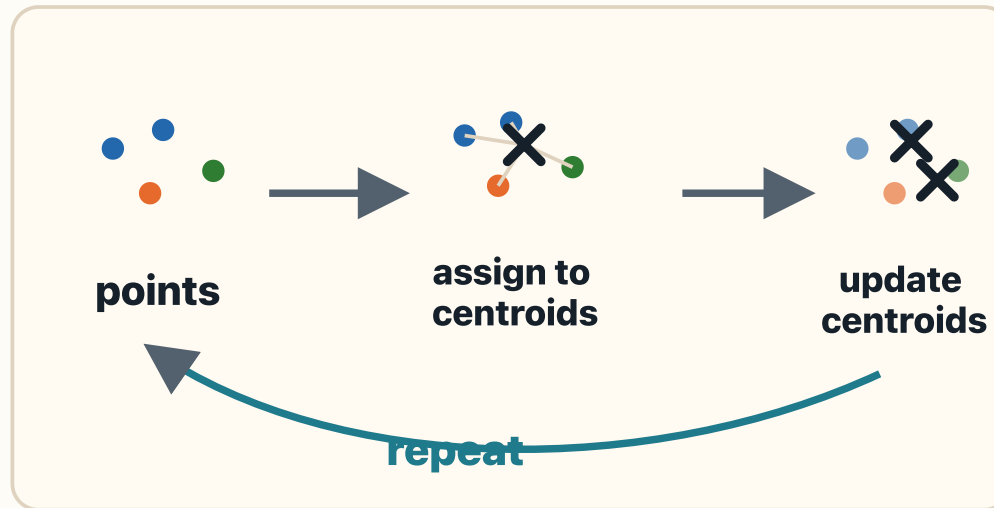
**Centroids move until no flower changes assignment.**

Species labels are context; k-means sees only measurements.

# Why k-means is a useful test case

A simple algorithm exposes the same pressures we see in many iterative methods.

## Iterative loop



**distance work per iteration**  
**roughly scales with  $N \times K \times d$**

## Validation target

same data  
same initial centroids  
same stopping rule  
comparable final inertia

## Harder data settings

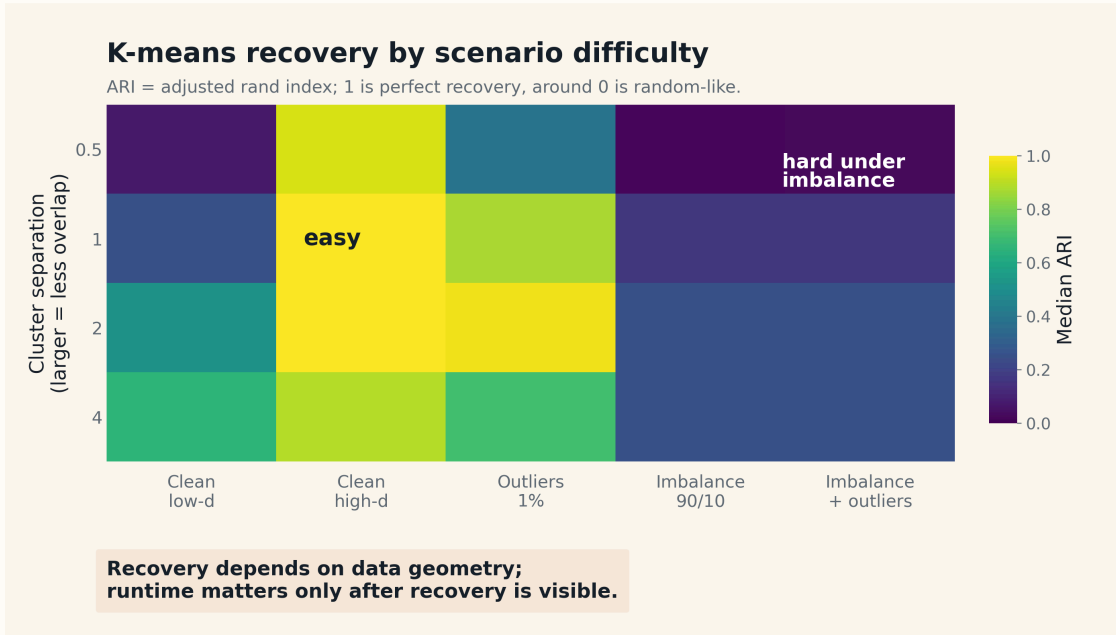
overlapping clusters  
imbalanced clusters  
outliers  
higher dimension

## Why Python tools matter

Numba: explicit loops  
NumPy: dense distance algebra  
JAX (A100): large regular array work

# First ask: did k-means recover the known simulated groups?

Simulation gives us the answer key: which points were generated from which group.



ARI = adjusted Rand index. 1 means perfect recovery; around 0 means random-like agreement with the simulated groups.

## What is being checked?

Do the fitted clusters match the known simulated groups?

## What changes by scenario?

Separation, dimension, outliers, and imbalance make recovery easier or harder.

## Why before timing?

If recovery is poor, speed is not evidence that the method worked.

**Benchmark speed only after the statistical result is acceptable.**

# Before timing: compare against the reference

Optimized k-means versions are only timed after they reproduce the same fixed-initialization result.

same data

same initial  
centroids

same stopping  
rule



**readable reference implementation**

the validation target before any speed claim



**Numba / NumPy distance algebra / JAX candidates**

optimized paths must preserve the same fixed-init result



**compare final inertia**

relative difference must stay below tolerance

MAIN CHECK

**3.1e-14**

max relative inertia  
difference

TOLERANCE

**1e-8**

Memory-risk reference rows are  
skipped explicitly, not hidden.

# How the same k-means step looks in Python

Same validation target; three ways to express the distance computation.

## Numba

Keep the assignment step as explicit loops.

```
@jit(parallel=True)
for i in prange(n):
    best = inf

    for k in range(K):
        dist = sqdist(X[i], C[k])
        if dist < best:
            best = dist
            labels[i] = k
```

**Shape:** loop-heavy CPU work without Python loop overhead.

## NumPy

Rewrite distances as dense matrix algebra.

```
x2 = sumsq(X)[: , None]
c2 = sumsq(C)[None, :]

dist2 = x2 + c2 - 2 * (X @ C.T)

labels = argmin(dist2, axis=1)
```

**Shape:** readable array expression; heavy matrix multiply is BLAS-backed.

## JAX (A100)

Use the same array expression on device.

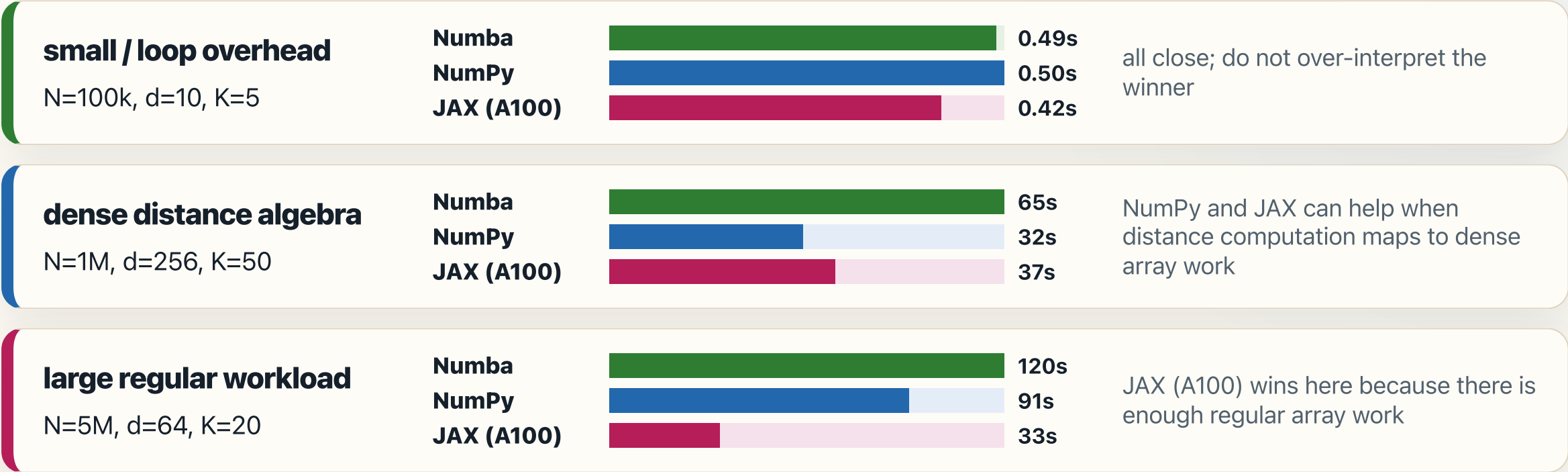
```
@jax.jit
def assign(X, C):
    x2 = sumsq(X)[: , None]
    c2 = sumsq(C)[None, :]

    dist2 = x2 + c2 - 2 * (X @ C.T)
    return jnp.argmin(dist2,
axis=1)
```

**Shape:** large regular array work, only useful after validation and batching.

**Same data. Same initialization. Same stopping rule. Then choose the expression that matches the bottleneck.**

# Server k-means: the best engine depends on the shape



These are representative validated rows, not a universal ranking.

# The tool choice follows the simulation signal

## Unstable recovery

Do not optimize yet; first interpret the method and scenario.

## Scalar loops

Use Numba to compile explicit CPU kernels.

## Dense distance algebra

Use NumPy when the rewrite removes large temporaries; its matrix multiply is BLAS-backed.

## Huge regular work

Use JAX (A100) only after enough batched work exists.

k-means stands in for a large class of iterative statistical algorithms: EM, coordinate descent, optimization loops, and simulation-based estimators.

## WORKLOAD 2

# Permutation tests as resampling inference pressure

---

A simple inference procedure becomes expensive when we need thousands of null draws across a high-dimensional features matrix.

# How a permutation test scales

One label shuffle gives one vector of feature-wise statistics; repeating shuffles builds the null distribution.

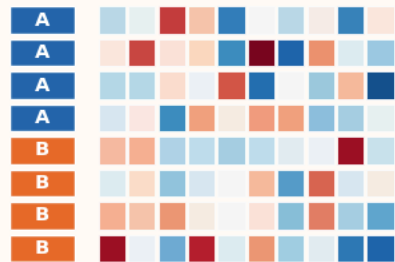
## 1. observed data

### X matrix

rows = samples columns = features

A/B labels

features

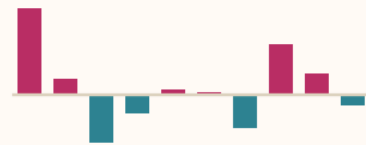


observed statistic  $T_{\text{obs}}$

## 2. one permutation

A A A A B B B B

compute group  
difference for  
every feature



one null draw = one vector  
of feature statistics

## 3. many permutations



### null-statistics collection

streamed exceedance counts: 0 4 1 5



contrast  
rows  $W$

→  $W @ X$

matrix path appears after repeat

## One resampling loop

1. shuffle group labels
2. compute feature-wise group differences
3. repeat to form the null distribution

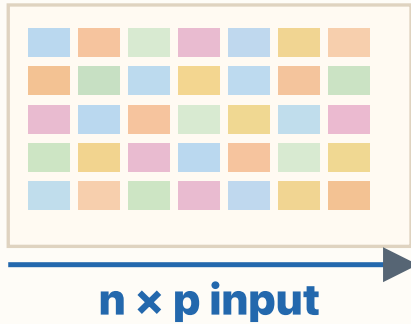
**Independent repetitions  
still create real work.**

# Why this becomes expensive

R permutations across p features means  $R \times p$  null statistics.

## X matrix

n rows = samples  
p columns = features

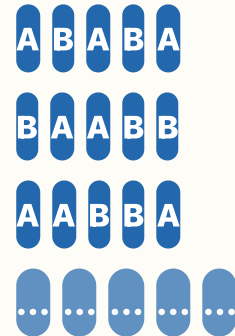


**samples** = patients  
**features** = genes / biomarkers / voxels



## R shuffled label vectors

R = null draws



**× R**



## R × p null statistics

or p exceedance counts after  
streamed reduction



**Simple statistic,  
large repeated  
workload.**

# Same permutations, different computation

Start with the readable loop; then rewrite the expensive repetition.

## Reference loop

```
for r in range(R):  
    labels_r = same_permutation_stream[r]  
    stat[r, :] = group_difference(X, labels_r)
```

## Batched / streamed path

```
for W_batch in contrast_batches(  
    same_permutation_stream):  
    T = W_batch @ X  
    counts += abs(T) >= abs(T_obs)
```

Same permutation stream. Same p-value definition. No full  $R \times p$  matrix materialized.

# Equivalence check: loop reference vs matrix formulation

Before timing the faster path, check that it answers the same statistical question.

## Same inputs

same simulated data  
same permutation stream  
same p-value definition

## Reference

readable loop formulation  
one permutation at a time

## Candidate

matrix formulation  
same stream, batched computation

## VALIDATION RESULT

# PASS

## COVERAGE

# 18 × 5

workloads × seeds

# 0.0

max |p diff|

# 9.4e-16

max |stat diff|

# 1e-6

validation  
tolerance

## Acceptance criteria

- ✓ p-values agree under the same permutation stream
- ✓ test statistics differ only at floating-point roundoff scale
- ✓ candidate path is allowed into runtime comparison

**Same question. Same result. Then faster code.**

# Null calibration gate: type-I error stays near alpha

Under the null, the type-I error should stay close to nominal  $\alpha = 0.05$ .

## Why calibration matters

Equivalence checks implementation. Calibration checks statistical behavior.

## Observed check

Observed type-I error = 0.051 at nominal alpha 0.050.

## Why timing now matters

The implementation and null behavior have both been checked.

**calibrated** **0.051**  
observed type-I error estimate

**0.050**

nominal alpha

**100**

null replicates

**500**

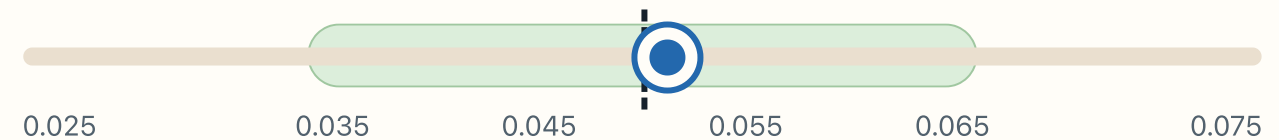
n

**1,000**

p and R

## Calibration ruler

feature-level binomial band 0.036-0.064



**Dashed:** nominal 0.05

**Dot:** observed mean 0.051

Band is feature-level binomial variation:  $0.05 \pm 1.96 * \sqrt{0.05 * 0.95 / 1000}$ .

**Speed matters only if null calibration remains correct.**

## LOCAL VALIDATION SCALE

# Local validation scale: implementations are close enough

### Scope

MacBook CPU validation tier; not a server/GPU leaderboard.

### Same permutations

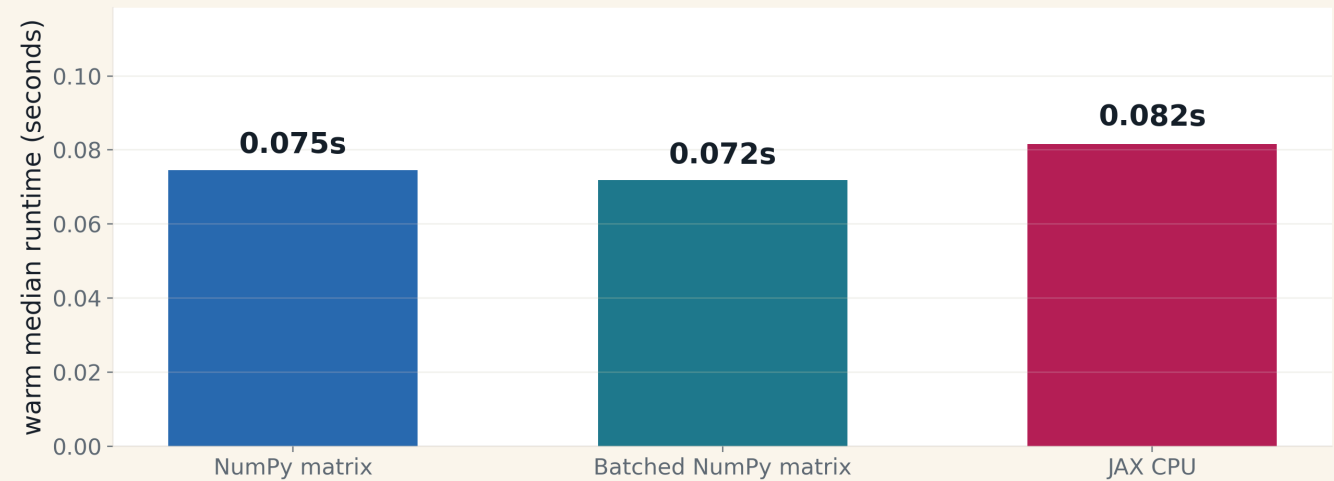
Compared only after p-value equivalence checks.

### Practical rule

At validation scale, readability and validation should dominate.

## Local validation scale: methods are close enough

MacBook CPU validation tier; n=500, p=1,000, R=1,000; same validated p-values.



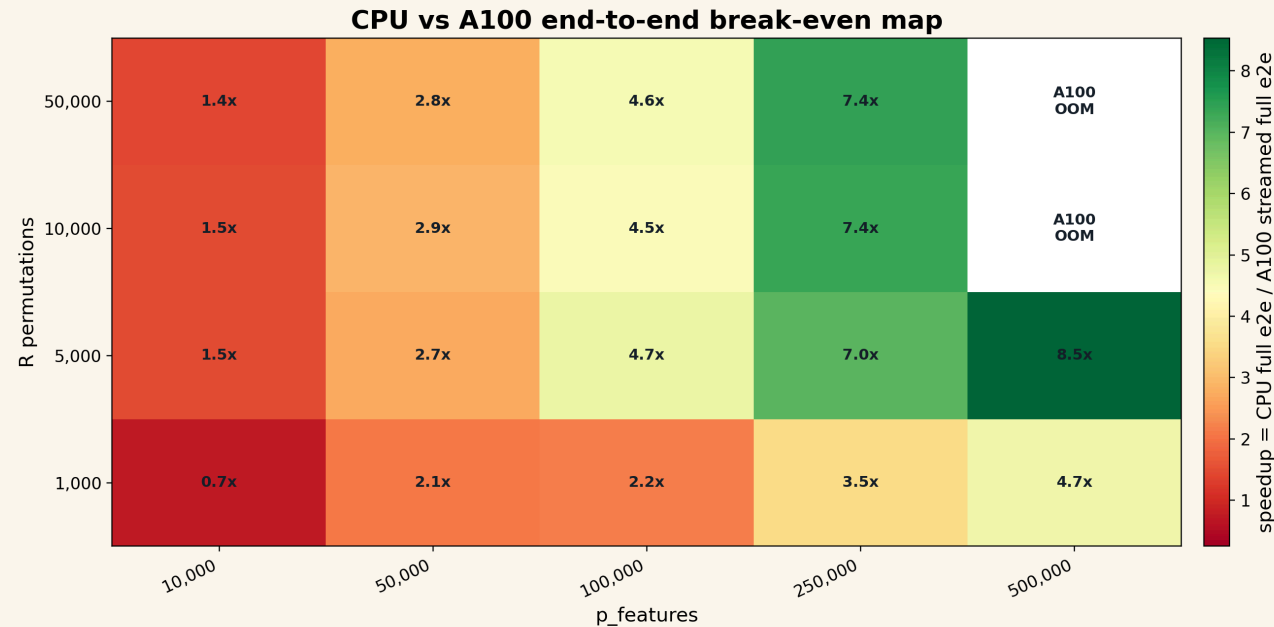
**Choose the clearest correct implementation until the bottleneck is real.**

**At validation scale, choose the clearest correct implementation; optimize after the bottleneck is real.**

## DECISION MAP

# When does GPU help for permutation tests?

A100 wins only after the permutation pipeline becomes large, batched, and device-resident enough.



**A100 faster region appears by  $n=5000$ ,  $p=10,000$ ,  $R=5,000$ ,  $\text{batch\_R}=8,192$ .**

### Timing scope

Compile excluded; transfer included; kernel-only excluded.

### Largest win

Largest slide-level measured speedup: 8.54x at  $n=5,000$ ,  $p=500,000$ ,  $R=5,000$ .

### A100 can win when

$R \times p$  is large,  $\text{batch\_R}$  fits memory, streamed reduction avoids collecting  $R \times p$ , and work stays on device.

speedup = matched CPU matrix baseline / A100 streamed full end-to-end.

**A100 becomes faster in the streamed-reduction grid at  $n=5,000$ ,  $p=10,000$ ,  $R=5,000$ ,  $\text{batch\_R}=8,192$ .**

# Where does A100 time go?

Full-scenario timing, not kernel-only timing: compile excluded, transfer included.

## Committed lightweight decomposition summary

Two representative rows; committed CSV has total time and W @ X share only.

### CPU-faster representative

n=5,000 · p=10,000 · R=1,000 · batch\_R=8,192

**0.24s** **19%** **0.70x**  
total W @ X speedup

W @ X

remaining full-scenario time not stage-resolved here

correctness: pass\_gpu\_tolerance · Detailed per-stage CSV not committed in this snapshot.

### Largest-speedup representative

n=5,000 · p=500,000 · R=5,000 · batch\_R=8,192

**1.25s** **19%** **8.54x**  
total W @ X speedup

W @ X

remaining full-scenario time not stage-resolved here

correctness: pass\_gpu\_tolerance · Detailed per-stage CSV not committed in this snapshot.

## Timing scope

Full scenario end-to-end. Compile excluded; transfer included. Kernel-only timing is excluded from speedup decisions.

## Missing detail

Only total time and W @ X share are committed here. This is a lightweight summary, not a full stage breakdown.

## Evidence scope

Committed representative summary shows 2 rows. Do not infer a four-row category set from uncommitted raw CSVs.

**Fast W @ X is necessary but not sufficient; the full statistical pipeline decides.**

## PARALLELISM

# CPU parallelism must be measured under resource constraints

Linux server CPU, controlled 1/4/16/64/128 sweep; high counts are shared-server evidence.

### k-means

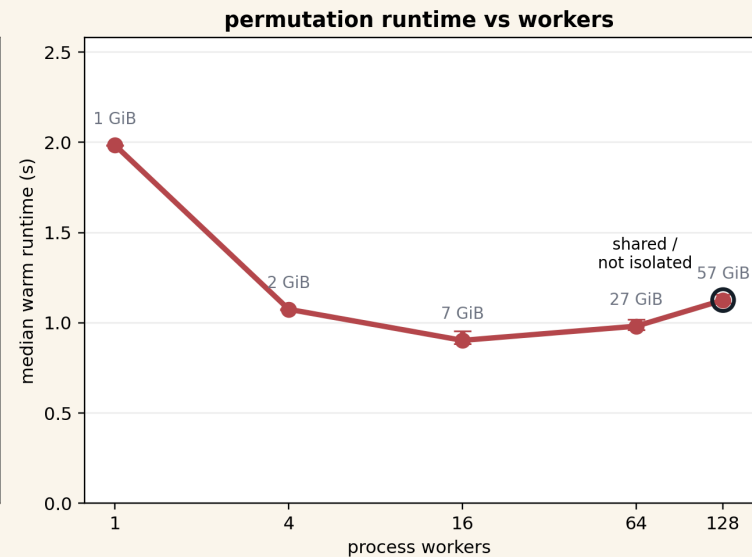
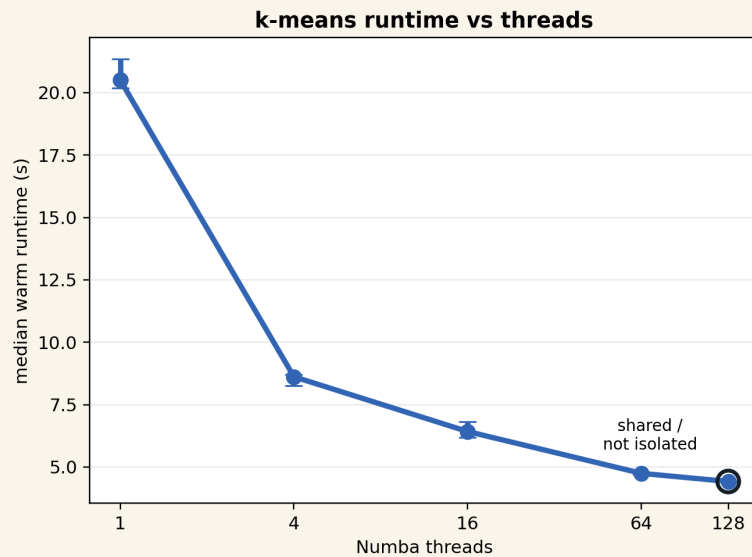
Numba kept improving through 128 in this shared-server run.

### permutation

Best runtime was at 16; memory rose to about 58 GiB at 128.

### 128 caveat

Affinity: 512 CPUs; no exclusive allocation.



High worker counts on a shared server are hard to interpret without CPU affinity and load checks.

# What each tool did in this talk

The tools are not a ranking; each one matches a different bottleneck shape.

## Numba

### Used for:

k-means assignment/update loops

Best fit: explicit numerical loops, low Python overhead, no giant temporary arrays.

## NumPy

### Used for:

dense distance algebra

Best fit: readable array expressions where the heavy matrix multiply is BLAS-backed.

## Threads / Python 3.14

### Used for:

repeated CPU work over shared arrays

Best fit: independent repetitions when memory and server resources are measured.

## JAX (A100)

### Used for:

streamed  $W @ X$  after validation

Best fit: large, regular, batched work that can stay device-friendly.

## Decision rule

- 1 Define the validation target.
- 2 Run validation checks against the readable reference.
- 3 Measure the bottleneck shape.
- 4 Choose the smallest tool that preserves the checked result.

## What not to infer

### Numba is not always faster.

It helps when the work is loop-shaped.

### More workers are not automatically better.

Memory, affinity, and server load matter.

### NumPy is not just plain Python.

Dense matrix work can call optimized BLAS underneath.

### JAX (A100) is not a free win.

Transfer, batching, and full-pipeline time decide.

**Same question. Same result. Then choose the smallest tool that matches the bottleneck.**

# Codex can generate variants; the statistician owns the scientific claims

## Automation layer

- implementation variants
- scenario-grid runners
- environment metadata
- plot regeneration
- README/result manifests

## Statistician decisions

- statistical target
- null model
- validation criteria
- interpretation of hard cases
- scientific claims

AI can automate experiment hygiene; it cannot define the scientific claim.

# What did the simulation reveal?

**Start from the validation  
result**

## **Validation fails**

keep reference / fix  
method

## **Hot scalar loop**

Numba

## **Dense matrix identity**

NumPy  
BLAS-backed

## **Repeated shared-array work**

threads / Python 3.14

## **Large batched device-resident work**

JAX (A100)

## **Giant temporary**

algebraic rewrite

CLOSE

# **Make the statistic testable. Then make the bottleneck fast.**

---

Clear enough to validate. Fast enough to scale.

`github.com/lucajiang/FastStatisticalModels4Python`

THANK YOU

# Thank you



Questions?

Clear enough to validate. Fast enough to scale.

`github.com/LucaJiang/FastStatisticalModels4Python`

## Evidence map: each environment answers a different question

### MacBook = validation

k-means equivalence, recovery surfaces, permutation p-value equivalence, calibration, and power.

### Server CPU = scale and parallelism

k-means shape scaling, Numba thread sweep, permutation R/p scaling, worker and memory sweeps.

### A100 = accelerator-shaped pipeline

JAX k-means break-even and streamed W @ X permutation follow-up.

Detailed row counts, skips, and status bookkeeping live in the result READMEs and speaker notes.

# Backup: Numba wins when k-means stays loop-shaped

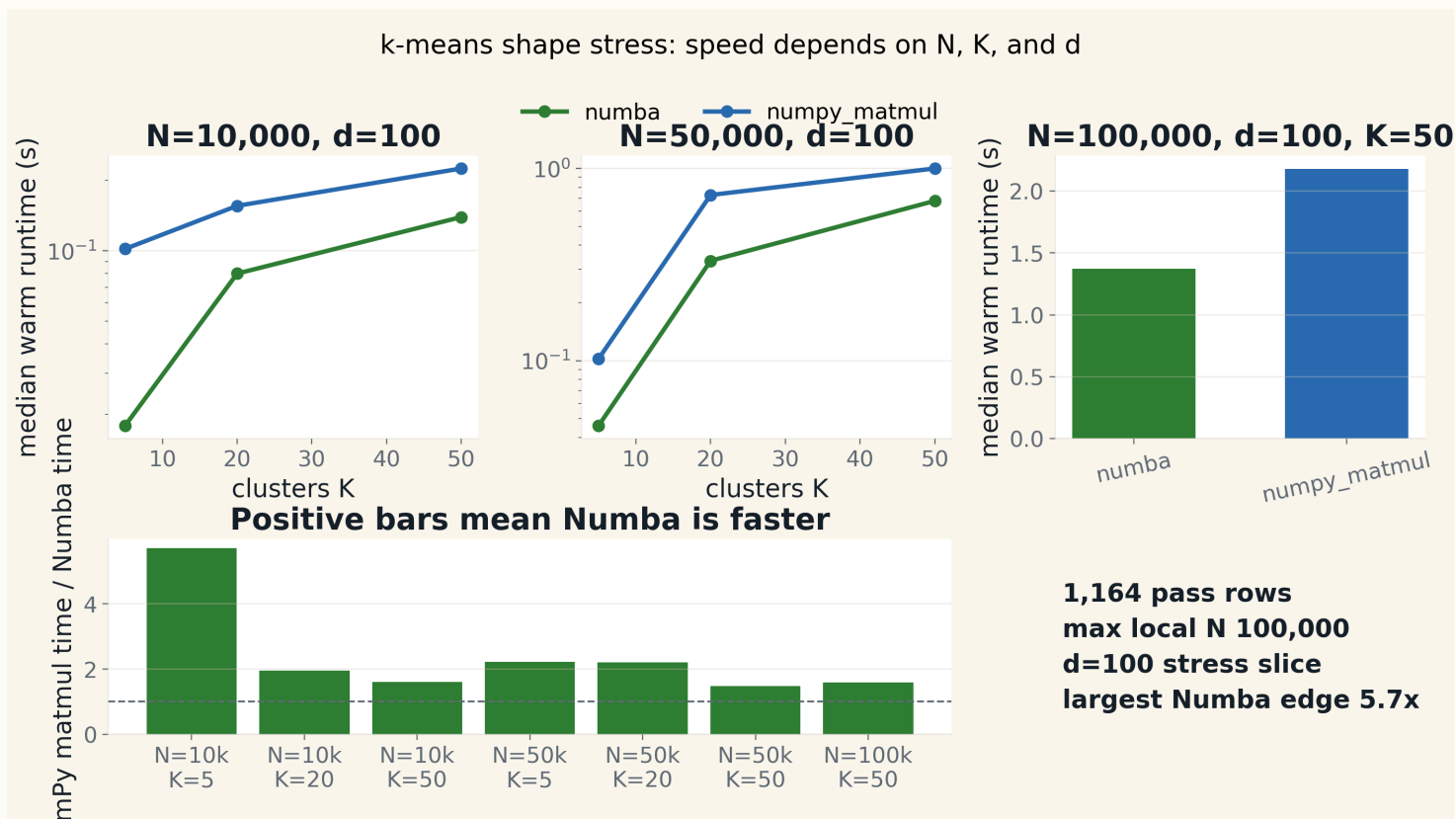
## Scope

**Local tier:** MacBook  
CPU validation-scale  
runs

**N** = samples, **d** =  
features, **K** = clusters

**Interpretation:** this slice  
is loop-shaped enough  
for Numba to win.

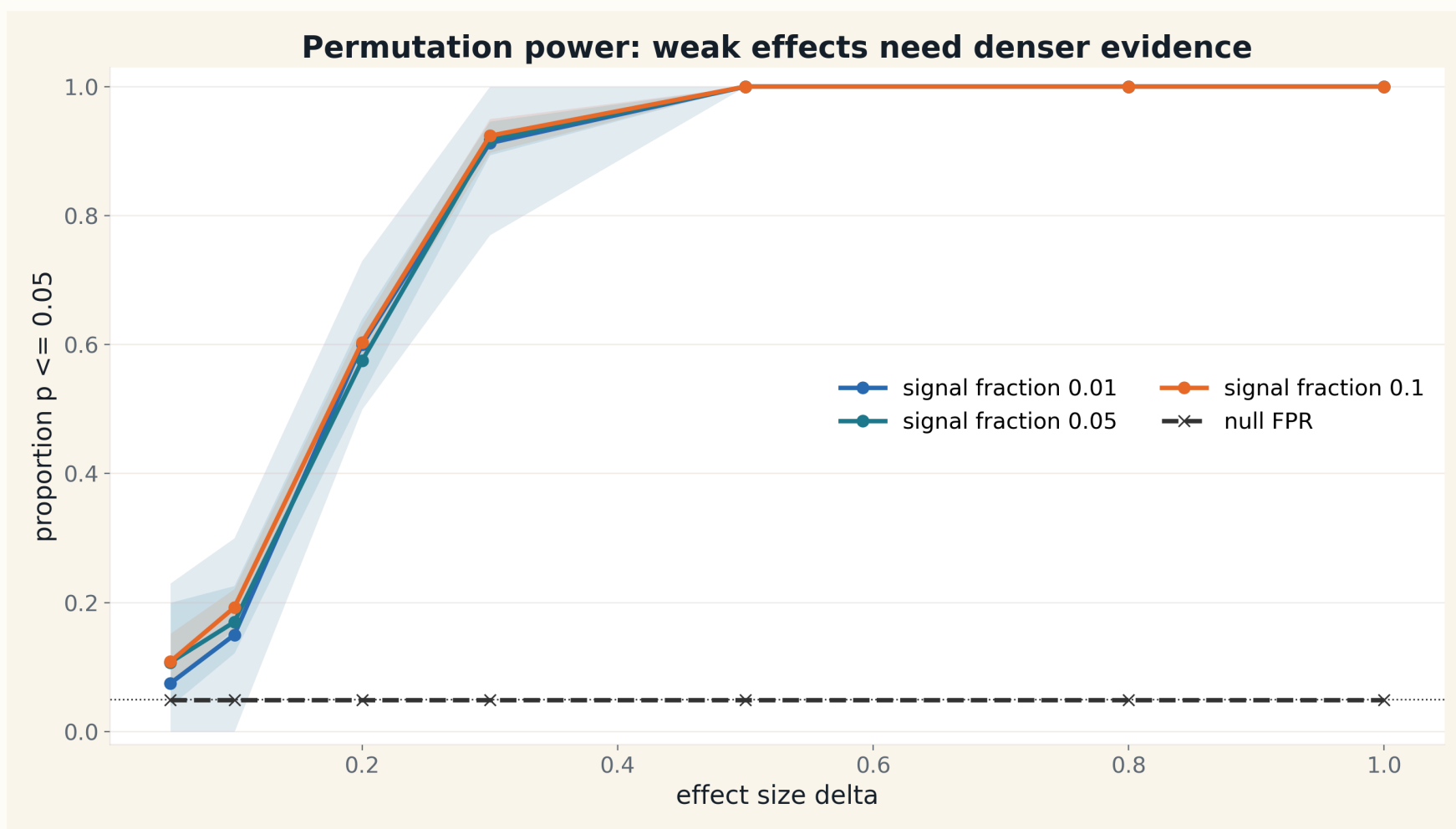
**Evidence:** 1,164 shape-  
stress rows, all pass;  
local loop-shaped  
evidence, not a universal  
ranking.



**1,164 pass rows**  
**max local N 100,000**  
**d=100 stress slice**  
**largest Numba edge 5.7x**

**This backup explains one branch of the rule: Numba wins when explicit loops dominate; NumPy and JAX (A100) can win when dense algebra or device-friendly scale dominates.**

## Power increases with effect size



Backup statistical sanity check: weak effects need more evidence; larger effects reach high power in this local curve.

# What the coding agent automated

## Experiment orchestration

Built resumable runners, scenario grids, environment capture, and result manifests.

## Visualization surface

Regenerated figures from CSVs and removed charts that looked precise but were unreadable.

## Repository surface

Kept reproducible commands, result summaries, and QA notes together.